

Induced Metric Optimisers

March 12, 2026

Contents

1	Problem Statement	2
2	Fixed Diagonal Metric	2
3	Learnable Scalar Metric	3
4	Learnable Diagonal Metric	6
5	Off-Diagonal Metric	9
6	Shared Implementation Details	12
7	Repository Structure	13
8	Results	14

1 Problem Statement

Given a model with N parameters $\theta(t) = \{\theta^i(t)\}_{i=1}^N$ and a loss function $\mathcal{L} : \mathbb{R}^N \rightarrow \mathbb{R}$, our goal is to find the optimal parameter update $\delta\theta$ at each time step t . That is, in preconditioned gradient flow we have

$$\frac{d\theta^i}{dt} = -g^{ij} \frac{\partial \mathcal{L}}{\partial \theta^j} \implies \delta\theta^i \approx -\eta g^{ij} \frac{\partial \mathcal{L}}{\partial \theta^j}$$

where $\eta \in \mathbb{R}_{>0}$ is the learning rate and g^{ij} is the inverse of the induced metric on the parameter space.

Across all variants in this document the induced metric is obtained by the same geometric construction: we embed parameter space into an ambient space via

$$\phi : \mathbb{R}^N \rightarrow \mathbb{R}^{N+1}, \quad \phi(\theta^1, \dots, \theta^N) = (\theta^1, \dots, \theta^N, f(\theta)),$$

where $f : \mathbb{R}^N \rightarrow \mathbb{R}$ is a scalar function of the parameters (taken to be the loss $\mathcal{L}$, or a function thereof), and then pull back an ambient bilinear form $h \in \mathbb{R}^{(N+1) \times (N+1)}$ to the parameter space. The variants differ in the choice of h (and, for the log-loss variant, the embedding function f ; see below).

We use the shorthand $l_i := \partial \mathcal{L} / \partial \theta^i$ for the raw gradient. When a base metric γ_{ij} is present, we write $l^i := \gamma^{ij} l_j$ for the metric-raised gradient (i.e. the inverse metric applied to $\mathbf{l}$). When $\gamma = \mathbb{1}_N$ (the identity), raising an index has no effect, so $l^i = l_i$ and the two notations coincide. The same index-raising convention applies to any covector: $a^i := \gamma^{ij} a_j$, $m^i := \gamma^{ij} m_j$, etc.

2 Fixed Diagonal Metric

The original ambient metric in the paper is

$$h = \begin{pmatrix} \gamma & \vec{0} \\ \vec{0}^\top & 1 \end{pmatrix} \in \mathbb{R}^{(N+1) \times (N+1)}$$

where $\gamma \in \mathbb{R}^{N \times N}$ is symmetric positive-definite (taken to be diagonal or scalar in all our experiments). The induced metric g on the parameter space is given by

$$g_{ij} = h_{\alpha\beta} \frac{\partial \phi^\alpha}{\partial \theta^i} \frac{\partial \phi^\beta}{\partial \theta^j} = \gamma_{ij} + \frac{\partial \mathcal{L}}{\partial \theta^i} \frac{\partial \mathcal{L}}{\partial \theta^j}.$$

Then, to calculate the parameter updates $\delta\theta^i$ we need g^{ij} , the inverse of g_{ij} . We can compute this inverse using the Sherman-Morrison-Woodbury formula:

Sherman-Morrison-Woodbury Formula

Let $A \in \mathbb{R}^{n \times n}$ be an invertible matrix, and let $U, V \in \mathbb{R}^{n \times k}$ be matrices. Then, if $C = A + UV^\top$ is invertible, we have

$$C^{-1} = A^{-1} - A^{-1}U(I_k + V^\top A^{-1}U)^{-1}V^\top A^{-1}.$$

This gives us

$$g^{ij} = \gamma^{ij} - \frac{\gamma^{ik}\gamma^{jl} \frac{\partial \mathcal{L}}{\partial \theta^k} \frac{\partial \mathcal{L}}{\partial \theta^l}}{1 + \gamma^{mn} \frac{\partial \mathcal{L}}{\partial \theta^m} \frac{\partial \mathcal{L}}{\partial \theta^n}} = \gamma^{ij} - \frac{l^i l^j}{1 + l_k l^k}$$

where

$$l_i := \frac{\partial \mathcal{L}}{\partial \theta^i} \implies l^i = \gamma^{ij} l_j.$$

Thus, the parameter updates are given by

$$\begin{aligned} \delta \theta^i &= -\eta \left(\gamma^{ij} - \frac{l^i l^j}{1 + l_k l^k} \right) l_j \\ &= -\eta l^i \left(1 - \frac{l_j l^j}{1 + l_k l^k} \right) \\ &= -\eta l^i \left(\frac{1}{1 + l_k l^k} \right). \end{aligned}$$

So to summarize, (1) we start with an ambient metric h which induces a metric g on the parameter space, (2) we compute the inverse g^{ij} , and (3) we use g^{ij} to compute the parameter updates $\delta \theta^i$.

Implementation

- JAX/Optax: `optimisers/jax.fixed.py`
- PyTorch: `optimisers/torch.fixed.py`
- Equivalence check: `optimisers/compare.fixed.py`

Exposed optimisers: `custom_sgd` (plain embedding, $f = \mathcal{L}$), `custom_sgd_log` (log-loss embedding, $f = \log \mathcal{L}$; requires $\mathcal{L} > 0$), `custom_sgd_rms` (RMS-normalised gradients, i.e. the update direction is $l_i/\|l\|$ instead of l_i). The sweep name is `sgd_metric / sgd_log_metric / sgd_rms`.

3 Learnable Scalar Metric

Construction

In the fixed diagonal variant, the base metric γ is a fixed hyperparameter. Here we instead allow γ to be *learned online* during optimisation, starting with the simplest possible form: a single scalar. We parameterise the inverse metric as

$$\gamma_{\phi}^{ij} = \alpha \delta^{ij}, \quad \alpha = e^s,$$

where $s \in \mathbb{R}$ is a single learnable scalar. Initialising $s = 0$ gives $\gamma^{-1} = \mathbb{1}_N$, so the optimiser starts as the unscaled fixed diagonal variant.

The ambient metric remains block-diagonal,

$$h = \begin{pmatrix} e^{-s} \mathbb{1}_N & \vec{0} \\ \vec{0}^\top & 1 \end{pmatrix},$$

and the induced metric on parameter space is

$$g_{ij} = e^{-s} \delta_{ij} + l_i l_j.$$

Applying the Sherman-Morrison-Woodbury formula with $A_{ij} = e^{-s} \delta_{ij}$, $u = v = \mathbf{l}$, we get

$$g^{ij} = e^s \delta^{ij} - \frac{e^{2s} l_i l_j}{1 + e^s \|\mathbf{l}\|^2}.$$

Here $\gamma_{ij} = e^{-s} \delta_{ij}$, so $l^i = \gamma^{ij} l_j = e^s \delta^{ij} l_j$ (identifying covectors and vectors via δ , this is just a scale factor). The parameter update is therefore

$$\delta \theta^i = -\eta g^{ij} l_j = -\eta l^i \left(1 - \frac{e^s \|\mathbf{l}\|^2}{1 + e^s \|\mathbf{l}\|^2} \right) = -\frac{\eta l^i}{1 + e^s \|\mathbf{l}\|^2}.$$

Let $\mathbf{l}_t = \nabla_{\theta} \mathcal{L}(\theta_t)$ denote the raw gradient vector at step t . Adding the hyperparameter ξ to control the overall strength of the metric correction, and using an exponential moving average (EMA) of the denominator quantity for stability (as in Algorithms 1 and 2 of the paper), the practical update is

$$\delta \theta^i = -\eta r_t (\alpha m^i), \quad r_t = \frac{1}{1 + |\hat{v}_t|}, \quad v_t = \xi \alpha \|\mathbf{l}_t\|^2, \quad (1)$$

where $\mathbf{m}$ is the bias-corrected momentum vector (in practice, the update direction $\mathbf{r}$ and the metric direction $\mathbf{l}$ need not coincide; see Section 6 for the full treatment) and $\hat{v}_t$ is the bias-corrected EMA of v_t . For the log-loss embedding (Algorithm 2, using $f = \log \mathcal{L}$ with $\mathcal{L} > 0$) the damping factor becomes $r_t = \mathcal{L}_t / (\mathcal{L}_t^2 + |\hat{v}_t|)$.

Mapping to Paper (Alg. 1/2)

- Inverse base metric: $\gamma^{-1} = \alpha \mathbb{1}_N = e^s \mathbb{1}_N$.
- Metric-raised gradient: $\tilde{l}_t^i = \gamma^{ij} l_{t,j} = \alpha \delta^{ij} l_{t,j}$.
- Trace statistic: $\tau_t = \xi \sum_i l_{t,i} \tilde{l}_t^i = \xi \alpha \|\mathbf{l}_t\|^2 = v_t$.
- Update direction: $\hat{m}_t^i = \gamma^{ij} m_{t,j} = \alpha m_t^i$.

(We write τ_t for the paper's trace scalar to avoid collision with the log-scale s .)

Metric Learning

We want to learn s so that $\alpha = e^s$ is calibrated to the local geometry of the loss. The trace quantity $v_t = \xi \alpha \|\mathbf{l}_t\|^2$ already captures how strongly the loss surface varies along the gradient direction under the current metric. We treat v_t as a surrogate objective and perform a *gradient ascent* step on s :

$$\frac{\partial v_t}{\partial s} = \xi e^s \|\mathbf{l}_t\|^2 = v_t.$$

Thus the online metric update is

$$s \leftarrow s + \mu v - \mu \lambda s \quad (2)$$

where $\mu > 0$ is the metric learning rate and $\lambda > 0$ is an L2 regularisation coefficient. The regularisation term $-\mu \lambda s$ penalises large deviations of s from zero (i.e. it pulls the inverse metric back towards the identity) and ensures stability. Finally s is clipped to the interval $[-s_{\max}, s_{\max}]$.

Intuitively: if the gradient is large, v is large, so s increases and α grows. A larger α amplifies the preconditioned direction $\alpha \mathbf{m}$ in the numerator, but also increases v (and hence $\hat{v}$) which shrinks r_t in the denominator. The net effect is that the metric adapts the scale of updates dynamically, with the regulariser providing a restoring force towards the identity.

Note that $v \geq 0$, so the unregularised ascent rule monotonically increases s . In practice this should be viewed as a stabilised feedback controller (regularisation + clipping + the denominator EMA) rather than an unbiased estimator of an optimal metric parameter.

Hyperparameters

The learnable scalar variant introduces three new hyperparameters beyond those of the fixed diagonal variant:

- μ (`metric_lr`): step size for the online metric update in (2).
- λ (`metric_reg`): L2 regularisation strength on s , controlling how far the metric drifts from identity.
- $s_{\max}$ (`metric_clip`): clipping bound on s , bounding $\alpha \in [e^{-s_{\max}}, e^{s_{\max}}]$.

Implementation

- JAX/Optax: `optimisers/jax_learnable_scalar.py`
- PyTorch: `optimisers/torch_learnable_scalar.py`
- Equivalence check: `optimisers/compare_learnable_scalar.py`

The plain variant is `custom_sgd_learnable_scalar` (sweep: `sgd_learn_scalar`). The log-loss variant is `custom_sgd_log_learnable_scalar` (sweep: `sgd_learn_scalar_log`).

The scalar metric state is a single float s stored per parameter group. At each step, s is updated *after* computing the parameter update using the same gradients (no separate forward pass):

1. Compute $v_t = \xi e^s \|\mathbf{l}_t\|^2$.
2. Update $s \leftarrow s + \mu v_t - \mu \lambda s$.
3. Clip $s \leftarrow \text{clip}(s, -s_{\max}, s_{\max})$.

4 Learnable Diagonal Metric

Construction

The learnable scalar metric uses a single degree of freedom and therefore cannot differentiate between parameters. We now promote s to a full vector $\mathbf{s} \in \mathbb{R}^N$ with one entry per parameter, giving the per-parameter inverse metric

$$\gamma_{\phi}^{ij} = e^{s_i} \delta^{ij}.$$

The ambient metric is

$$h = \begin{pmatrix} \text{diag}(e^{-\mathbf{s}}) & \vec{0} \\ \vec{0}^\top & 1 \end{pmatrix},$$

and the induced metric on parameter space is

$$g_{ij} = e^{-s_i} \delta_{ij} + l_i l_j.$$

This is again a rank-1 update to a diagonal matrix. Applying Sherman-Morrison-Woodbury with $A_{ij} = e^{-s_i} \delta_{ij}$ and $u = v = \mathbf{l}$,

$$g^{ij} = e^{s_i} \delta^{ij} - \frac{e^{s_i} l_i e^{s_j} l_j}{1 + \sum_k e^{s_k} l_k^2}.$$

Writing $l^i = \gamma^{ij} l_j = e^{s_i} \delta^{ij} l_j$, the update is

$$\delta \theta^i = -\eta g^{ij} l_j = -\eta \left(l^i - \frac{l^i \sum_k e^{s_k} l_k^2}{1 + \sum_k e^{s_k} l_k^2} \right) = -\frac{\eta l^i}{1 + \sum_k e^{s_k} l_k^2}.$$

With ξ , momentum $\mathbf{m}$, and the EMA damping, the practical update is

$$\delta \theta^i = -\eta r_t (e^{s_i} m^i), \quad r_t = \frac{1}{1 + |\hat{v}_t|}, \quad v_t = \xi \sum_k e^{s_k} l_k^2, \quad (3)$$

with the same log-loss variant as before. Compared to the scalar case, each parameter θ^i now has an individual scale e^{s_i} , allowing the metric to adapt separately to different regions of parameter space.

Mapping to Paper (Alg. 1/2)

- Inverse base metric: $\gamma^{ij} = e^{s_i} \delta^{ij}$.
- Metric-raised gradient: $\tilde{l}_t^i = \gamma^{ij} l_{t,j} = e^{s_i} \delta^{ij} l_{t,j}$.
- Trace statistic: $\tau_t = \xi \sum_k l_{t,k} \tilde{l}_t^k = \xi \sum_k e^{s_k} l_{t,k}^2 = v_t$.
- Update direction: $\hat{m}_t^i = \gamma^{ij} m_{t,j} = e^{s_i} m_t^i$.

(We write τ_t for the paper’s trace scalar to avoid collision with the log-scales s_i .)

Metric Learning

The per-parameter metric update treats each s_i independently. The surrogate objective is

$$J(\mathbf{s}) = \xi \sum_k e^{s_k} l_k^2,$$

and its gradient with respect to s_i is

$$\frac{\partial J}{\partial s_i} = \xi e^{s_i} l_i^2.$$

The online update is therefore

$$s_i \leftarrow s_i + \mu \xi e^{s_i} l_i^2 - \mu \lambda s_i \quad (4)$$

followed by mean-centering and clipping (discussed below).

Implementation note. The current diagonal implementations omit the factor of ξ in the s -update, effectively absorbing it into μ . The scalar variant implementations do include ξ . The formulas above reflect the mathematically consistent version; reconciling the code is tracked as a known issue.

Mean-Centering

After the gradient step (4), we apply a per-leaf mean-centering operation (i.e. independently for each parameter tensor in the model):

$$s_i \leftarrow s_i - \bar{s}, \quad \bar{s} = \frac{1}{N_\ell} \sum_{k=1}^{N_\ell} s_k,$$

where N_ℓ is the number of entries in the current parameter tensor (leaf), not the total model size. This is not an ad-hoc trick but a principled normalisation. The trace $v = \xi \sum_k e^{s_k} l_k^2$ has a global scale degeneracy: shifting all s_k by a

constant c is equivalent to scaling ξ by e^c and leaving $\mathbf{s}$ unchanged. Concretely, writing $s_k = \bar{s} + \delta_k$ (so $\sum_k \delta_k = 0$),

$$v = \xi e^{\bar{s}} \sum_k e^{\delta_k} l_k^2.$$

The global mean $\bar{s}$ is exactly degenerate with ξ . Mean-centering enforces $\bar{s} = 0$ after centering (note: subsequent clipping may re-introduce a small nonzero mean), which fixes this gauge and ensures that ξ remains the single scalar controlling the overall metric magnitude while $\mathbf{s}$ captures only the *relative* per-parameter geometry. Since we apply this per leaf rather than globally, a separate mean mode is removed in each parameter tensor. This is a stronger constraint than global gauge-fixing: it prevents a leaf-wise uniform rescaling mode, not just a model-wide one.

Fixed Diagonal as a Special Case

In a highly symmetric special case: if all s_i are initialised to the same value and the gradients have the same magnitude for all parameters at every step, then $\mathbf{s}$ stays uniform and mean-centering keeps it at zero. In that case $e^{s_i} = 1$ for all i and $v = \xi \|\mathbf{l}\|^2$, which reduces exactly to the fixed diagonal variant with $\gamma = \mathbb{1}_N$. The learnable diagonal metric therefore generalises fixed diagonal by learning *relative* per-parameter scalings, while mean-centering intentionally removes the global scale mode (which is degenerate with ξ). To recover a learnable global scale, disable mean-centering or make ξ itself learnable.

Implementation

- JAX/Optax: `optimisers/jax_learnable_diag.py`
- PyTorch: `optimisers/torch_learnable_diag.py`
- Equivalence check: `optimisers/compare_learnable_diag.py`

The plain variant is `custom_sgd_learnable_diag` (sweep: `sgd_learn_diag`). The log-loss variant is `custom_sgd_log_learnable_diag` (sweep: `sgd_learn_diag_log`).

The diagonal metric state is a tensor $\mathbf{s}$ with the same shape as the parameters, initialised to zero. At each step:

1. Compute $v = \xi \sum_k e^{s_k} l_k^2$ (scalar denominator trace).
2. Update $s_i \leftarrow s_i + \mu \xi e^{s_i} l_i^2 - \mu \lambda s_i$ for each i .
3. Mean-centre per leaf: $s_i \leftarrow s_i - \bar{s}$.
4. Clip $\mathbf{s} \leftarrow \text{clip}(\mathbf{s}, -s_{\max}, s_{\max})$.

As with the scalar case, both the metric update and the parameter update use the same gradients within a single step.

5 Off-Diagonal Metric

Now, we consider a more general ambient bilinear form of the form

$$h = \begin{pmatrix} \gamma & \vec{a} \\ \vec{b}^\top & 1 \end{pmatrix} \in \mathbb{R}^{(N+1) \times (N+1)}$$

where $\vec{a}, \vec{b} \in \mathbb{R}^N$. Intuitively, the vectors $\vec{a}$ and $\vec{b}$ introduce off-diagonal couplings between the parameter space and the loss function dimension. In general, if $\vec{a} \neq \vec{b}$ this h is not symmetric, so here we simply treat it as a preconditioning matrix rather than a Riemannian metric.

Stability note. If the effective preconditioner is non-symmetric ($\vec{a} \neq \vec{b}$), the update direction need not be a descent direction even for convex objectives. We recommend defaulting to the symmetric case $\vec{a} = \vec{b}$; if using $\vec{a} \neq \vec{b}$, consider symmetrising the effective preconditioner or otherwise enforcing positive-definiteness. The derivation below sets $\xi = 1$ for clarity; the implementations include the general ξ scaling factor, replacing $g = \gamma + UV^\top$ with $g = \gamma + \xi UV^\top$. The induced bilinear form on the parameter space is then given by (with $\xi = 1$)

$$\begin{aligned} g_{ij} &= h_{\alpha\beta} \frac{\partial \phi^\alpha}{\partial \theta^i} \frac{\partial \phi^\beta}{\partial \theta^j} \\ &= \gamma_{ij} + a_i \frac{\partial \mathcal{L}}{\partial \theta^j} + b_j \frac{\partial \mathcal{L}}{\partial \theta^i} + \frac{\partial \mathcal{L}}{\partial \theta^i} \frac{\partial \mathcal{L}}{\partial \theta^j} \\ &= \gamma_{ij} + a_i l_j + b_j l_i + l_i l_j. \end{aligned}$$

To compute the inverse g^{ij} , we notice that g can be expressed as a rank-2 update to γ , that is for $U = [a, l], V = [l, b + l] \in \mathbb{R}^{N \times 2}$ we have

$$\begin{aligned} \gamma_{ij} + U_{ik} V_{jk} &= \gamma_{ij} + a_i l_j + (b_j + l_j) l_i \\ &= \gamma_{ij} + a_i l_j + b_j l_i + l_i l_j \\ &= g_{ij}. \end{aligned}$$

Thus, we can apply the Sherman-Morrison-Woodbury formula again to get

$$\begin{aligned} g^{ij} &= \gamma^{ij} - \gamma^{ik} U_{k\alpha} \left(\delta_{\alpha\beta} + V_{m\alpha} \gamma^{mn} U_{n\beta} \right)^{-1} V_{l\beta} \gamma^{lj} \\ &= \gamma^{ij} - \gamma^{ik} (a_k, l_k) \begin{pmatrix} 1 + l_n a^n & l_n l^n \\ (b_m + l_m) \gamma^{mn} a_n & 1 + (b_m + l_m) \gamma^{mn} l_n \end{pmatrix}^{-1} \begin{pmatrix} l_n \\ b_n + l_n \end{pmatrix} \gamma^{nj}. \end{aligned}$$

For convenience we define the following scalar contractions:

$$c_{al} := a_n l^n, \quad c_{ll} := l_n l^n, \quad c_{ba} := b_n a^n, \quad c_{bl} := b_n l^n.$$

Thus, we have

$$\begin{aligned} g^{ij} &= \gamma^{ij} - \frac{\gamma^{ik}}{D} \left[a_k ((1 + c_{bl}) l_n - c_{ll} b_n) \right. \\ &\quad \left. + l_k (-(c_{ba} + c_{al}) l_n + (1 + c_{al})(b_n + l_n)) \right] \gamma^{nj} \end{aligned}$$

where

$$\begin{aligned} D &= (1 + c_{al})(1 + c_{bl} + c_{ul}) - c_{ul}(c_{ba} + c_{al}) \\ &= 1 + c_{al} + c_{bl} + c_{al}c_{bl} + c_{ul} - c_{ul}c_{ba}. \end{aligned}$$

The closed-form inverse requires $D \neq 0$ (equivalently, $\det(\mathbb{1}_2 + V^\top \gamma^{-1} U) \neq 0$). The JAX implementation includes a near-singular fallback; the PyTorch implementation solves the 2×2 system directly via `torch.linalg.solve`.

A numerical check of this has been performed in `documents/off-diag-check.ipynb`: for random $\vec{a}, \vec{b}, \gamma, \vec{l}$ the closed-form $g^{ij}l_j$ matches `torch.linalg.solve(g, l)` within floating-point tolerance.

Quick Sanity Check

If we set $\vec{a} = \vec{0}$ and $\vec{b} = \vec{0}$, we should recover the diagonal metric case. In this case, $c_{al} = 0$, $c_{ba} = 0$, $c_{bl} = 0$, and hence $D = 1 + c_{ul}$. Then

$$g^{ij} = \gamma^{ij} - \gamma^{ik} \frac{1}{1 + c_{ul}} l_k l_n \gamma^{nj} = \gamma^{ij} - \frac{l^i l^j}{1 + l_k l^k},$$

which matches our previous result.

If we set $\vec{a} = \vec{0}$ but keep $\vec{b} \neq \vec{0}$, then $c_{al} = 0$, $c_{ba} = 0$, and $D = 1 + c_{bl} + c_{ul}$. The update reduces to $\delta\theta^i = -\eta l^i / D$, i.e. a scaled gradient step whose denominator depends on $\vec{b}$ through c_{bl} . This confirms that $\vec{b}$ affects the effective learning rate even when $\vec{a} = \vec{0}$.

The parameter updates are given by

$$\delta\theta^i = -\eta g^{ij} l_j = -\eta \left[\frac{1 + c_{al}}{D} l^i - \frac{c_{ul}}{D} a^i \right].$$

The update direction now has two components: one along l^i (the gradient direction) and one along a^i (the off-diagonal coupling direction). Depending on the choices of $\vec{a}$ and $\vec{b}$, this can lead to a variety of different optimization behaviors:

- If $\vec{a} = \vec{0}$ or $\vec{a} \propto \vec{l}$, we have plain gradient descent with an adaptive learning rate.
- If $\vec{a} = \text{momentum}$, we have a momentum-like update with an adaptive learning rate.
- If $\vec{a} = \vec{\theta}$ (the parameter vector), we have an update that combines gradient descent with a form of weight decay or growth depending on the sign of c_{ul}/D .

For $\vec{a} = \vec{b} = \vec{l}$ the update simplifies to

$$\delta\theta^i = -\eta \frac{l^i}{1 + 3c_{ul}}$$

which is similar to the diagonal case but with a factor of 3 in the denominator (corresponding to an effective $\xi = 3$ in the $\xi = 1$ normal form used above).

Implementation

- JAX/Optax: `optimisers/jax_offdiag.py`
- PyTorch: `optimisers/torch_offdiag.py`
- Equivalence check: `optimisers/compare_offdiag.py`

Sweep names follow the pattern `sgd_offdiag_{a}_{b}` where `{a}` and `{b}` are drawn from `0` (zero), `1` (gradient), `m` (momentum), `theta` (parameters). All pairs except `sgd_offdiag_0_0` (which reduces to the fixed diagonal variant) are registered in `parameters/optimizer_registry.py`.

Computational cost. Taking $\gamma_{ij} = \gamma \delta_{ij}$ (scalar base metric) and writing

$$G = \gamma \mathbb{1}_N + \xi UV^\top, \quad U = (\mathbf{a}, \mathbf{l}), \quad V = (\mathbf{l}, \mathbf{b} + \mathbf{l}),$$

the Sherman-Morrison-Woodbury formula gives

$$G^{-1}\mathbf{r} = \frac{1}{\gamma}\mathbf{r} - \frac{\xi}{\gamma^2}U\left(\mathbb{1}_2 + \frac{\xi}{\gamma}V^\top U\right)^{-1}V^\top\mathbf{r}.$$

This costs $O(N)$: only inner products and a single 2×2 matrix inversion are required, never a full $N \times N$ matrix.

Modes for $\mathbf{l}$, $\mathbf{a}$, and $\mathbf{b}$. The vector $\mathbf{l}$ encodes the direction along which the loss coordinate stretches, with two built-in choices:

- **Gradient mode:** $\mathbf{l}_i = \partial_i \mathcal{L}$ — the metric is shaped by the instantaneous slope.
- **Momentum mode:** $\mathbf{l}_i = \mathbf{m}_i$ — the metric incorporates accumulated gradient history.

The off-diagonal coupling vectors $\mathbf{a}$ and $\mathbf{b}$ each support:

- **Zero:** $\mathbf{a}_i = 0$ or $\mathbf{b}_i = 0$ — eliminates one branch of the coupling.
- **Gradient:** aligned with $\partial_i \mathcal{L}$.
- **Momentum:** aligned with $\mathbf{m}_i$.
- **Parameter:** $\mathbf{a}_i = \theta^i$ or $\mathbf{b}_i = \theta^i$ — couples geometry to position.
- **Same-as-a:** $\mathbf{b}_i = \mathbf{a}_i$ — symmetric off-diagonal structure.

For full generality the implementations also accept user-provided vectors or callables.

6 Shared Implementation Details

Update Direction vs. Metric Direction

In all implementations, the goal is to realise

$$\delta\theta^i = -\eta g^{ij} r_j,$$

where r_j is the update direction and l_i is the direction used to construct the metric. These need not be the same vector. In ordinary gradient descent one would take $r_j = l_j = \partial_j \mathcal{L}$, but we also test momentum-based choices. The four combinations are:

- $l = \nabla \mathcal{L}$, $r = \nabla \mathcal{L}$: pure geometric gradient descent.
- $l = \nabla \mathcal{L}$, $r = \mathbf{m}$: metric shaped by the instantaneous gradient, step along momentum.
- $l = \mathbf{m}$, $r = \nabla \mathcal{L}$: metric shaped by accumulated history, step along raw gradient.
- $l = \mathbf{m}$, $r = \mathbf{m}$: both metric and step are momentum-based.

Computational Cost

All variants are $O(N)$ per step (assuming scalar or diagonal γ) and never materialise a full $N \times N$ matrix:

Variant	Dominant operation	Additional metric state
Fixed diagonal	scalar $\times$ vector	none
Learnable scalar	scalar $\times$ vector	1 float (s)
Learnable diagonal	element-wise vector $\times$ vector	N floats ($\mathbf{s}$)
Off-diagonal	inner products + 2×2 inverse	none

The metric-learning overhead for the learnable variants is negligible compared to the forward and backward passes.

Shared Hyperparameters

The following hyperparameters are common to the fixed diagonal, learnable scalar, and learnable diagonal variants:

- η (**lr**): base learning rate.
- β_m (**momentum**): EMA decay for the momentum buffer. (The paper uses μ for this quantity; we write β_m to avoid collision with the metric learning rate μ .)
- ξ (**xi**): strength of the induced metric correction.

- β (**beta**): EMA decay for the denominator estimate $\hat{v}$.
- λ_θ (**weight_decay**): decoupled weight decay (AdamW style).

The off-diagonal variant uses η , β_m , ξ , and λ_θ but does not use the denominator EMA β . The learnable variants additionally require:

- μ (**metric_lr**): step size for the online metric update.
- λ (**metric_reg**): L2 regularisation on s or $\mathbf{s}$.
- $s_{\max}$ (**metric_clip**): log-domain clipping bound on s , constraining $\alpha = e^s \in [e^{-s_{\max}}, e^{s_{\max}}]$.

7 Repository Structure

Directory Layout

Directory	Contents
<code>optimisers/</code>	JAX and PyTorch optimiser implementations and equivalence checks
<code>parameters/</code>	Hyperparameter sweep scripts and shared training utilities
<code>analysis/</code>	Jupyter notebooks for loading and visualising sweep results
<code>documents/</code>	This document and supplementary notebooks
<code>images/</code>	Figures used in the paper and this document
<code>results/</code>	Local sweep results (JSON, gitignored)
<code>data/</code>	Local data files (gitignored)

Running Sweeps

Hyperparameter sweeps are driven by scripts in `parameters/` and support two backends: **local** (Optuna + JSON files, no account required) and **WandB** (cloud logging).

Script	Task	Model
<code>sweep_mnist_mlp.py</code>	MNIST classification	MLP (64 hidden)
<code>sweep_cifar10_resnet18.py</code>	CIFAR-10 classification	ResNet-18
<code>sweep_regression.py</code>	Synthetic regression	MLP
<code>sweep_shake.py</code>	Shakespeare language model	MiniGPT
<code>sweep_small_examples.py</code>	2D test functions	n/a

All scripts share a common CLI:

- `--optimiser`: optimiser name (see below).
- `--num_runs`: number of Optuna trials (default: 50).

- `--backend`: local or wandb (default: wandb).
- `--index`: run index for organising multiple runs of the same sweep.

Results are written to `results/<task>/<optimiser>/run_<index>/<run_id>.json`, each containing the full training history, final summary metrics, and the hyperparameter config used.

Available Optimisers

Sweep name	Description
adam, adamw, sgd, muon	Baselines
sgd_metric, sgd_log_metric, sgd_rms	Fixed diagonal metric
sgd_learn_scalar, sgd_learn_scalar_log	Learnable scalar metric
sgd_learn_diag, sgd_learn_diag_log	Learnable diagonal metric
sgd_offdiag-{a}-{b}	Off-diagonal metric ($\{0, l, m, \theta\}^2$ pairs, excluding 0_0)

The full optimiser registry is `parameters/optimizer_registry.py`.

Analysis Notebooks

Notebook	Task
analysis/small_examples.ipynb	2D test functions (Beale, Rosenbrock, etc.)
analysis/mnist_analysis.ipynb	MNIST MLP
analysis/cifar_analysis.ipynb	CIFAR-10 ResNet-18
analysis/regression_analysis.ipynb	Synthetic regression
analysis/shakespeare_analysis.ipynb	Shakespeare language modelling

Each notebook supports both backends. Configure `BACKEND`, `TASK_TAG`, and `OPTIMIZERS` in the second cell. Outputs include training curves, 2D histograms of best metric vs. epoch, time-to-best analysis, and best-hyperparameter tables (also saved as CSV to `analysis/`).

8 Results

All optimisers are evaluated together on the same suite of benchmark tasks. Results are compared head-to-head in the following notebooks:

- `analysis/small_examples.ipynb` — 2D synthetic functions (Beale, Rosenbrock, Himmelblau, Ackley, Rastrigin).

